

PrazaShop

“A túa praza, no teu móbil”

<https://github.com/Abrahamfdez/PrazaShop-app>



Nome Alumno/a: Abraham Fernandez Bande

Curso: 2º DAM

Módulo: Proxecto Final Ciclo

Contido

Índice

Contido.....	2
1. Introducción.....	3
2. Obxectivos.....	3
3. Situación previa.....	4
4. Tecnoloxías empregadas.....	5
5. Solución proposta.....	6
6. Planificación do proxecto.....	7
6.1. Diagrama de Gantt.....	7
6.2. Descrición do modelo de desenvolvemento de software a implementar.....	7
6.3. Diagrama de casos de usos.....	8
6.4. Universo de discurso da base de datos.....	9
6.5. Diagrama de Entidad-Relación.....	10
6.6. Deseño do proxecto:.....	10
7. Desenvolvemento e execución.....	23
8. Conclusións e reflexións.....	23
9. Bibliografía e Webgrafía.....	25
10. Anexo I – Manual técnico de instalación ou posta en marcha.....	27
11. Anexo II – Documentación de uso (manuais usuario).....	31
12. Anexo III – Outra documentación.....	38

1. Introducción

O presente proxecto consiste no desenvolvemento dunha aplicación móbil orientada a dar visibilidade o comercio local da miña cidade, Ourense.

Esta app permite que os negocios publiquen os seus produtos e os clientes poidan realizar e valorar as compras en ditos comercios.

Os clientes ademais, poderán crear compras recorrentes fomentando o consumo de produtos de tempada e alimentos de calidade.

A súa vez os negocios poderán anunciar a duración do produto que están ofertando (Solo hoxe, Esta semana ou durante a tempada deste produto).

2. Obxectivos

Obxectivo Principal

Desenvolver unha app que permita aos negocios locais publicar os seus produtos e os clientes consúltalos de maneira sinxela.

Obxectivos Específicos

- Crear unha app multiplataforma con flutter:
 - o Un dos obxectivos principais é crear unha aplicación móbil multiplataforma utilizando Flutter. Esta tecnoloxía permite desenvolver aplicacións que poden executarse en diferentes sistemas operativos a partir dun único código fonte, reducindo así o tempo de desenvolvemento.
- Realizar unha Api Rest en java utilizando Spring:
 - o Outro dos obxectivos do proxecto é desenvolver unha API REST utilizando Java e o framework Spring Boot. Esta API será a encargada de xestionar toda a lóxica do sistema. A API permitirá a comunicación entre a aplicación móbil e a base de datos, asegurando unha arquitectura organizada e escalable.

- Realizar unha base de datos relacional en mySQL:
 - o O proxecto incluírá tamén o deseño e implementación dunha base de datos relacional utilizando MySQL. Nesta base de datos almacenaranse os datos da aplicación, como información sobre os usuarios, os produtos publicados polos negocios ou as compras realizadas polos clientes.
- Implementar un sistemas de compras recorrentes:
 - o Unha das funcionalidades máis interesantes da aplicación será a posibilidade de que os usuarios poidan crear compras recorrentes. Isto permitirá que determinados produtos se poidan pedir automaticamente cada certo tempo, por exemplo semanalmente.
- Desenvolver un sistema de Ranking para os negocios por parte dos consumidores:
 - o Outro dos obxectivos do proxecto é implementar un sistema de valoración ou ranking que permita aos usuarios avaliar os negocios ou os produtos adquiridos.

3. Situación previa

Na actualidade, moitos comercios locais enfróntanse a unha perda de visibilidade fronte ás grandes cadeas comerciais e ás plataformas dixitais de venda online.

Mentres que as grandes empresas dispoñen de aplicacións móbiles, páxinas web e sistemas de venda dixital, moitos pequenos negocios continúan dependendo principalmente da clientela habitual ou da visibilidade física do establecemento.

Ademais, moitos destes comercios son negocios familiares ou tradicionais que non sempre dispoñen dos coñecementos técnicos necesarios para desenvolver ferramentas dixitais propias.

Ao mesmo tempo, os hábitos de consumo dos clientes cambiaron nos últimos anos. Cada vez é máis habitual que os usuarios busquen produtos ou servizos utilizando o teléfono móbil.

Neste contexto, resulta necesario desenvolver ferramentas tecnolóxicas que permitan aos pequenos negocios mellorar a súa presenza dixital e facilitar o acceso aos seus produtos.

4. Tecnoloxías empregadas

A aplicación estará dividida en varias capas principais: o **frontend**, encargado da interface de usuario, o **backend**, responsable da lóxica da aplicación e da xestión de datos, e finalmente a **base de datos**, onde se almacenará toda a información do sistema.

Frontend: Flutter

Unha das súas principais vantaxes é que permite desenvolver aplicacións multiplataforma cun único código fonte, o que significa que a mesma aplicación pode executarse en diferentes sistemas operativos.

Backend: Java con SpringBoot

Para o backend utilízase Java xunto co framework Spring Boot. Esta combinación permite desenvolver aplicacións web e APIs REST de maneira rápida e estruturada.

Spring Boot simplifica a configuración das aplicacións e proporciona ferramentas para xestionar a comunicación entre o servidor e os clientes, así como para interactuar coa base de datos.

Base de datos: MySQL

MySQL é un sistema de xestión de bases de datos relacional amplamente utilizado en aplicacións web e móbiles.

No proxecto PrazaShop utilízase para almacenar toda a información da aplicación, como usuarios, produtos, pedidos ou valoracións.

Ferramentas de desenvolvemento:

- **Git** utilizarase para o control de versións do código fonte.
- **GitHub** permitirá almacenar o repositorio e colaborar no desenvolvemento do proxecto
- **IntelliJ IDEA** empregarase principalmente para o desenvolvemento do backend en Java.
- **Visual Studio Code** utilizarase para traballar co código do frontend.
- **Postman** permitirá probar e validar as chamadas á API REST durante o desenvolvemento.

5. Solución proposta

A solución proposta consiste no desenvolvemento dunha aplicación móbil chamada PrazaShop que permitirá conectar aos consumidores cos comercios locais.

A aplicación permitirá que os negocios publiquen os seus produtos indicando información como o prezo, a dispoñibilidade ou a duración da oferta/producto.

Os usuarios poderán navegar polos produtos dispoñibles, consultar información sobre os comercios e realizar pedidos desde o seu teléfono móbil.

Ademais, a aplicación incluírá funcionalidades como a creación de compras recorrentes e un sistema de valoración dos negocios por parte dos usuarios.

A principal diferenza desta solución respecto a outras plataformas de comercio electrónico é que está centrada especificamente no comercio local e nos produtos frescos ou artesanais.

6. Planificación do proxecto

6.1. Diagrama de Gantt

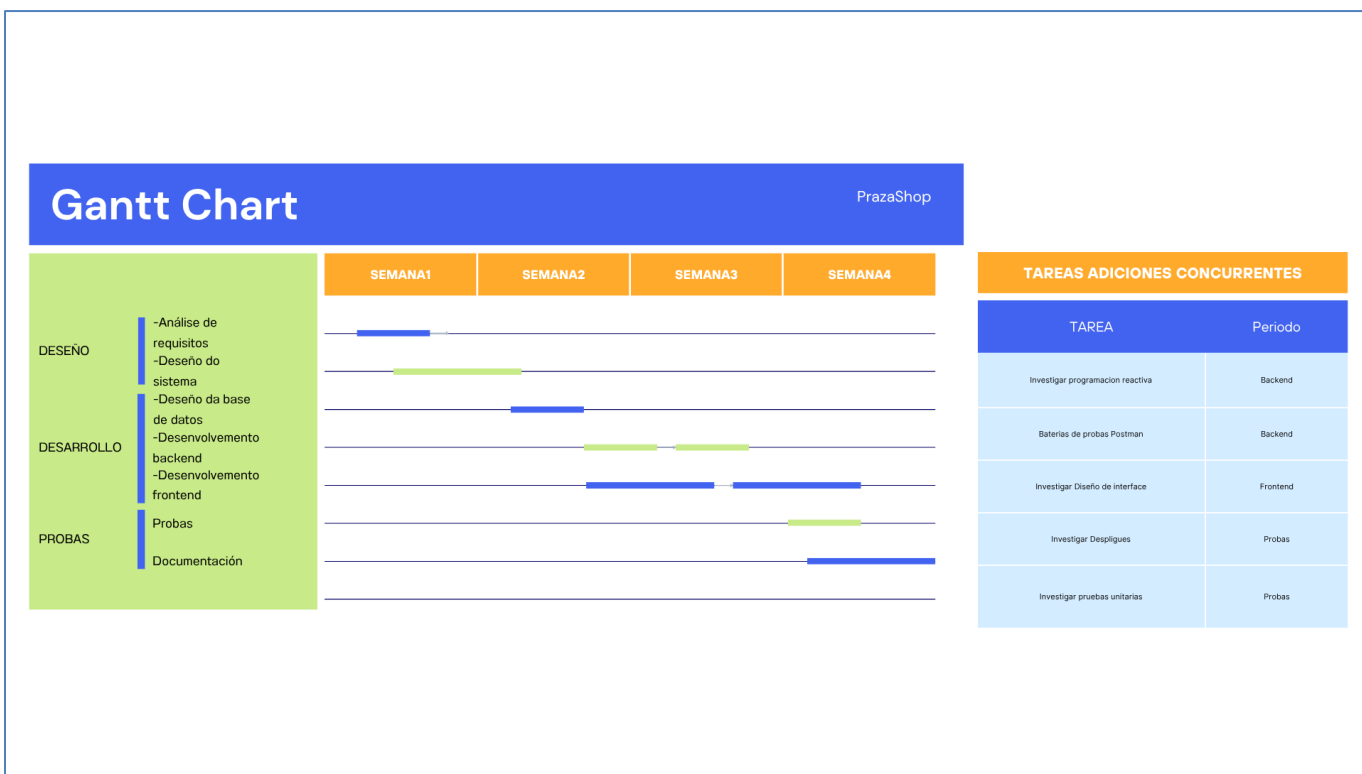


Tabla 1-Diagrama de Gantt

6.2. Descripción do modelo de desenvolvemento de software a implementar.

Para o desenvolvemento do proxecto PrazaShop optouse por un modelo incremental e iterativo.

Este modelo baséase na construción do sistema mediante sucesivas versións funcionais, nas que se van engadindo novas características de forma progresiva.

O empezar a fase de desarrollo o noso proxecto vai pasar por 3 fases:

- DEV
- PRE
- PRO

Este enfoque permite detectar erros de forma temperá, mellorar a calidade do sistema e adaptar o desenvolvemento ás necesidades que poidan xurdir durante o proceso.

6.3. Diagrama de casos de usos.



Ilustración 1 Casos de Uso

6.4. Universo de discurso da base de datos.

O sistema **PrazaShop** está orientado á xestión dunha plataforma que conecta clientes con negocios locais, permitindo a consulta, compra e xestión de produtos dispoñibles na aplicación.

Na plataforma existen diferentes tipos de usuarios: **clientes, negocios e administradores**. Todos eles comparten información básica como nome, correo electrónico, contrasinal, teléfono e tipo de usuario. A partir desta entidade xeral, os clientes e os negocios dispoñen de información específica segundo o seu papel dentro do sistema.

Os **clientes** poden consultar os produtos publicados polos negocios, realizar pedidos, configurar compras recorrentes e valorar os negocios. Cada cliente está asociado a un usuario da plataforma e pode contar cunha dirección de envío para recibir os seus pedidos.

Os **negocios** representan establecementos locais rexistrados na aplicación. Cada negocio está asociado tamén a un usuario e almacena información propia como o nome do negocio, a dirección e unha descrición do establecemento. Un negocio pode publicar múltiples produtos e recibir pedidos realizados polos clientes.

Os **produtos** son publicados polos negocios e conteñen información como o nome, descrición, prezo, stock dispoñible, categoría, duración da oferta, imaxe e estado. Cada produto pertence a un único negocio, mentres que un negocio pode ter varios produtos asociados.

As compras realízanse mediante **pedidos**. Cada pedido está asociado a un cliente e a un negocio, e inclúe información como a data do pedido, datas de confirmación, entrega ou cancelación, o estado do pedido e o importe total. Para representar os produtos incluídos nun pedido utilízase unha entidade de **detalle de pedido**, onde se almacena cada produto comprado, a cantidade solicitada e o prezo unitario no momento da compra.

Ademais, os clientes poden configurar **compras recorrentes**, que permiten automatizar pedidos periódicos dun mesmo produto.

O sistema tamén permite que os clientes realicen **valoracións** dos negocios. Cada valoración está asociada a un cliente e a un negocio, e inclúe unha puntuación, un comentario opcional e a data na que foi realizada.

Para a autenticación e seguridade da aplicación, a base de datos conta cunha entidade de **tokens**, encargada de almacenar os tokens de acceso ou refresco asociados aos usuarios, así como o seu estado de revogación ou expiración.

Finalmente, a plataforma incorpora unha entidade de **movementos de stock**, destinada á auditoría e control das modificacións no stock dos produtos. Estes movementos permiten rexistrar operacións como reservas, confirmacións, liberacións ou axustes manuais, podendo estar asociados a un produto e, nalgúns casos, a un pedido concreto.

6.5. Diagrama de Entidad-Relación.

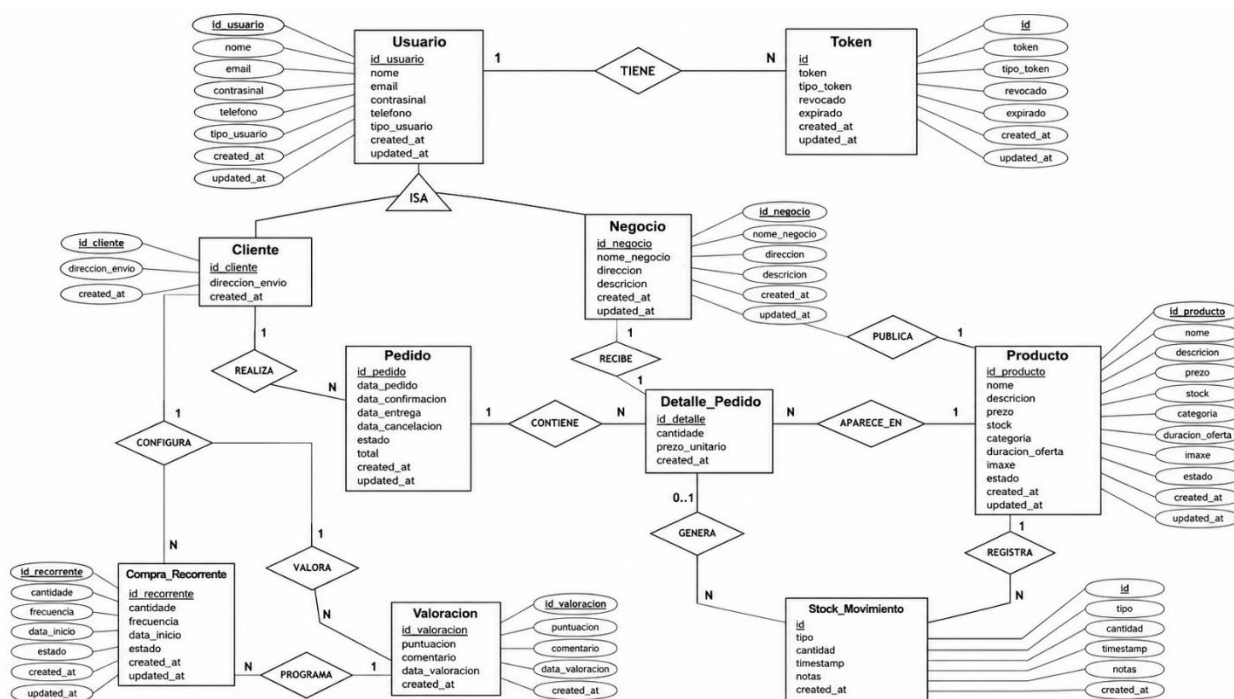


Ilustración 2 E:R CHEN

6.6. Deseño do proxecto:

Modelo relacional da base de datos.

O modelo relacional da base de datos de **PrazaShop** define as táboas necesarias para almacenar a información da aplicación e as relacións existentes entre elas. Este modelo deriva do diagrama entidade-relación e permite representar de forma estruturada os usuarios, clientes, negocios, produtos, pedidos, detalles de pedido, compras recorrentes, valoracións, tokens de autenticación e movementos de stock.

Relacións do modelo

```

1  USUARIO (
2    id_usuario      PK,
3    nome,
4    email,
5    contrasinal,
6    telefono,
7    tipo_usuario,
8    created_at,
9    updated_at
10 );
11
12 CLIENTE (
13   id_cliente      PK,
14   id_usuario      FK,
15   direccion_envio,
16   created_at
17 );
18
19 -- FK: id_usuario → USUARIO(id_usuario)
20
21
22 NEGOCIO (
23   id_negocio      PK,
24   id_usuario      FK,
25   nome_negocio,
26   direccion,
27   descripcion,
28   created_at,
29   updated_at
30 );
31
32 -- FK: id_usuario → USUARIO(id_usuario)
33

```

```

1  PRODUCTO (
2    id_producto    PK,
3    id_negocio     FK,
4    nome,
5    descripcion,
6    prezo,
7    stock,
8    categoria,
9    duracion_oferta,
10   imaxe,
11   estado,
12   created_at,
13   updated_at
14 );
15
16 -- FK: id_negocio → NEGOCIO(id_negocio)
17
18
19 PEDIDO (
20   id_pedido      PK,
21   id_cliente     FK,
22   id_negocio     FK,
23   data_pedido,
24   data_confirmacion,
25   data_entrega,
26   data_cancelacion,
27   estado,
28   total,
29   created_at,
30   updated_at
31 );
32
33 -- FK: id_cliente → CLIENTE(id_cliente)
34 -- FK: id_negocio → NEGOCIO(id_negocio)
35
36
37 DETALLE_PEDIDO (
38   id_detalle     PK,
39   id_pedido      FK,
40   id_producto    FK,
41   cantidade,
42   prezo_unitario,
43   created_at
44 );
45
46 -- FK: id_pedido → PEDIDO(id_pedido)
47 -- FK: id_producto → PRODUCTO(id_producto)

```



```
1  COMPRA_RECURRENTE (
2      id_recorrente      PK,
3      id_cliente         FK,
4      id_producto        FK,
5      cantidade,
6      frecuencia,
7      data_inicio,
8      estado,
9      created_at,
10     updated_at
11 );
12
13 -- FK: id_cliente → CLIENTE(id_cliente)
14 -- FK: id_producto → PRODUCTO(id_producto)
15
16
17 VALORACION (
18     id_valoracion      PK,
19     id_cliente         FK,
20     id_negocio         FK,
21     puntuacion,
22     comentario,
23     data_valoracion,
24     created_at
25 );
26
27 -- FK: id_cliente → CLIENTE(id_cliente)
28 -- FK: id_negocio → NEGOCIO(id_negocio)
29
30
31 TOKEN (
32     id                  PK,
33     token,
34     tipo_token,
35     revocado,
36     expirado,
37     id_usuario         FK,
38     created_at,
39     updated_at
40 );
41
42 -- FK: id_usuario → USUARIO(id_usuario)
43
44
45 STOCK_MOVIMIENTO (
46     id                  PK,
47     id_producto        FK,
48     id_pedido          FK NULL,
49     tipo,
50     cantidad,
51     timestamp,
52     notas,
53     created_at
54 );
55
56 -- FK: id_producto → PRODUCTO(id_producto)
57 -- FK: id_pedido → PEDIDO(id_pedido)
```

Descrición do modelo

A táboa **USUARIO** almacena a información común de acceso á aplicación, tanto para clientes como para negocios e administradores. Inclúe datos como o nome, correo electrónico, contrasinal, teléfono, tipo de usuario e datas de creación e actualización.

A partir da táboa **USUARIO** relaciónanse as táboas **CLIENTE** e **NEGOCIO**. Cada cliente ou negocio está asociado a un único usuario da plataforma.

A táboa **NEGOCIO** almacena a información propia dos establecementos locais, como o nome do negocio, a dirección e a descrición. Un negocio pode publicar varios produtos na plataforma, polo que se relaciona coa táboa **PRODUCTO**.

A táboa **PRODUCTO** almacena os produtos publicados polos negocios. Cada produto contén información como nome, descrición, prezo, stock dispoñible, categoría, duración da oferta, imaxe, estado e datas de creación e actualización.

A táboa **PEDIDO** representa as compras realizadas polos clientes. Cada pedido está asociado a un cliente e a un negocio, e inclúe información como a data do pedido, a data de confirmación, entrega ou cancelación, o estado do pedido e o importe total.

Cada pedido descomponse na táboa **DETALLE_PEDIDO**, onde se almacenan os produtos concretos incluídos en cada pedido, xunto coa cantidade solicitada e o prezo unitario no momento da compra.

A táboa **COMPRA_RECORRENTE** permite gardar as compras programadas polos clientes, indicando o produto, a cantidade, a frecuencia, a data de inicio e o estado da compra recorrente.

A táboa **VALORACION** recolle as puntuacións e comentarios que os clientes realizan sobre os negocios. Cada valoración pertence a un cliente e a un negocio.

A táboa **TOKEN** almacena os tokens empregados para a autenticación da aplicación. Cada token está asociado a un usuario e indica o tipo de token, se foi revogado ou se está expirado.

Por último, a táboa **STOCK_MOVIMIENTO** permite rexistrar os movementos de stock dos produtos. Estes movementos poden corresponder a reservas, confirmacións, liberacións ou axustes manuais. Cada movemento está asociado a un produto e, opcionalmente, a un pedido.

Cardinalidades principais



```
1  USUARIO 1:1 CLIENTE
2  USUARIO 1:1 NEGOCIO
3  USUARIO 1:N TOKEN
4
5  NEGOCIO 1:N PRODUCTO
6  NEGOCIO 1:N PEDIDO
7  NEGOCIO 1:N VALORACION
8
9  CLIENTE 1:N PEDIDO
10 CLIENTE 1:N COMPRA_RECURRENTE
11 CLIENTE 1:N VALORACION
12
13
14 PEDIDO 1:N DETALLE_PEDIDO
15 PRODUCTO 1:N DETALLE_PEDIDO
16
17 PRODUCTO 1:N COMPRA_RECURRENTE
18 PRODUCTO 1:N STOCK_MOVIMIENTO
19
20 PEDIDO 0:N STOCK_MOVIMIENTO
21 No caso de PEDIDO 0:N STOCK_MOVIMIENTO,
22 a relación é opcional porque un movemento de stock
23 pode non estar asociado a ningún pedido,
24 por exemplo cando se trata dun axuste manual de stock.
```

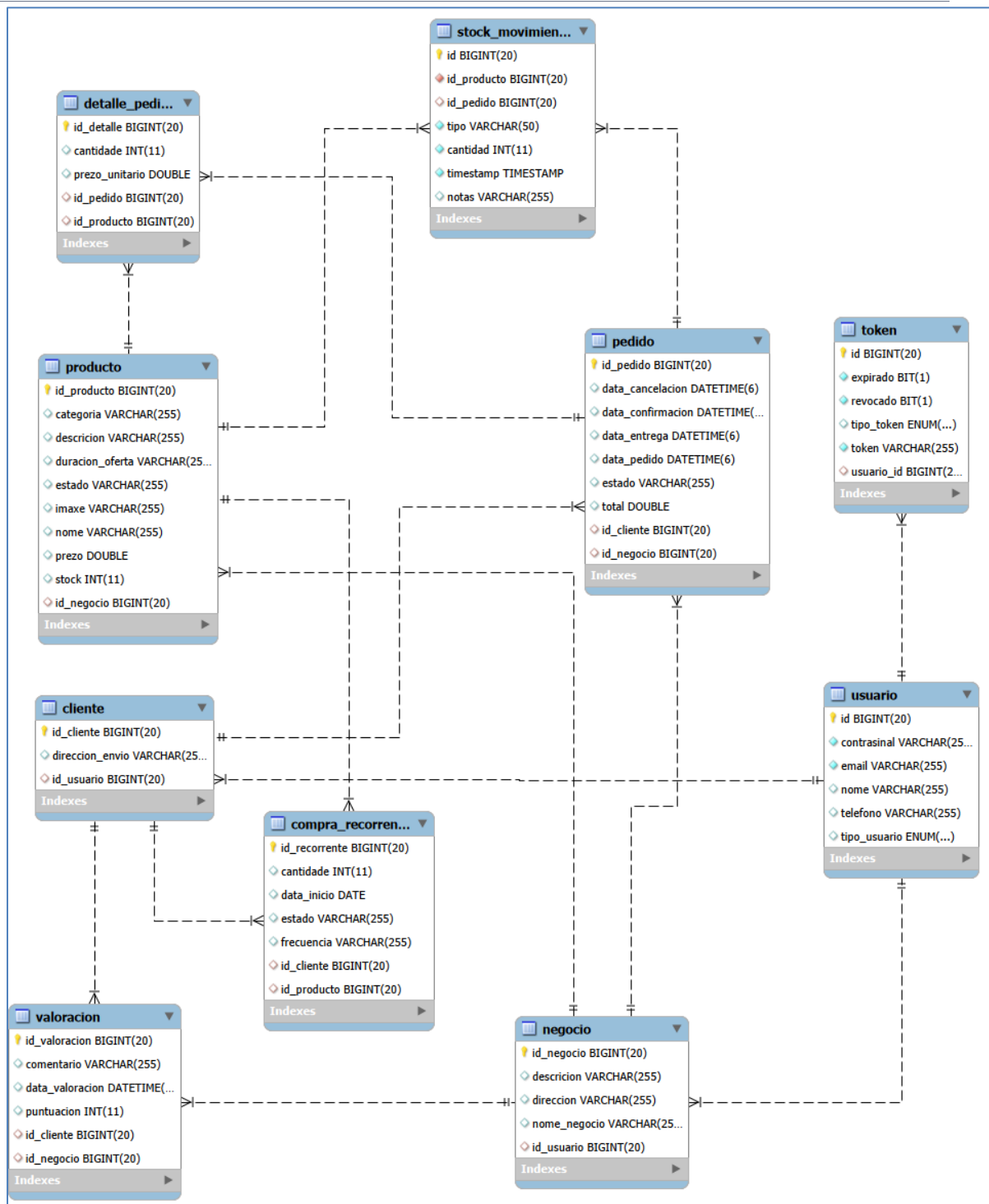


Ilustración 3 E:R MySQL SCHEMA

Diagrama de clases.

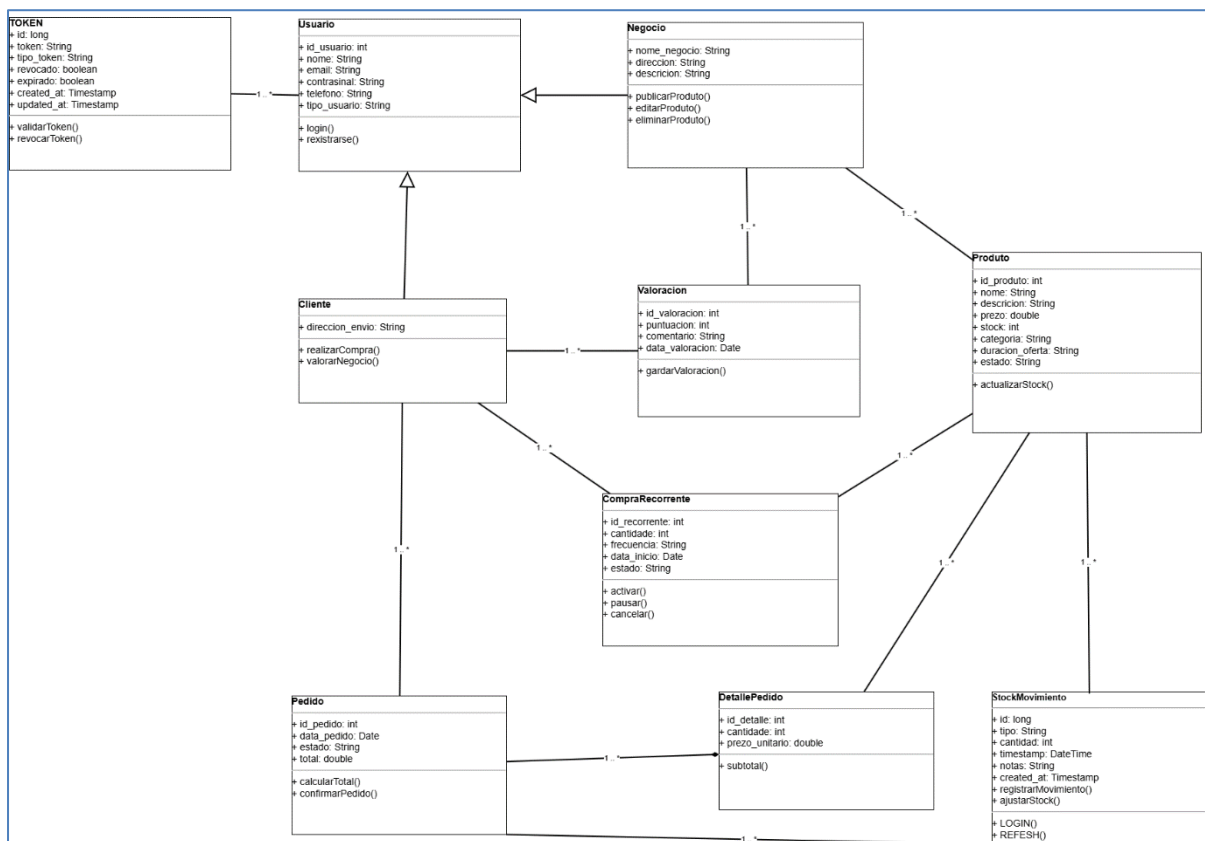


Ilustración 4 DIAGRAMA CLASES

Diagramas de fluxo de cada caso de uso.

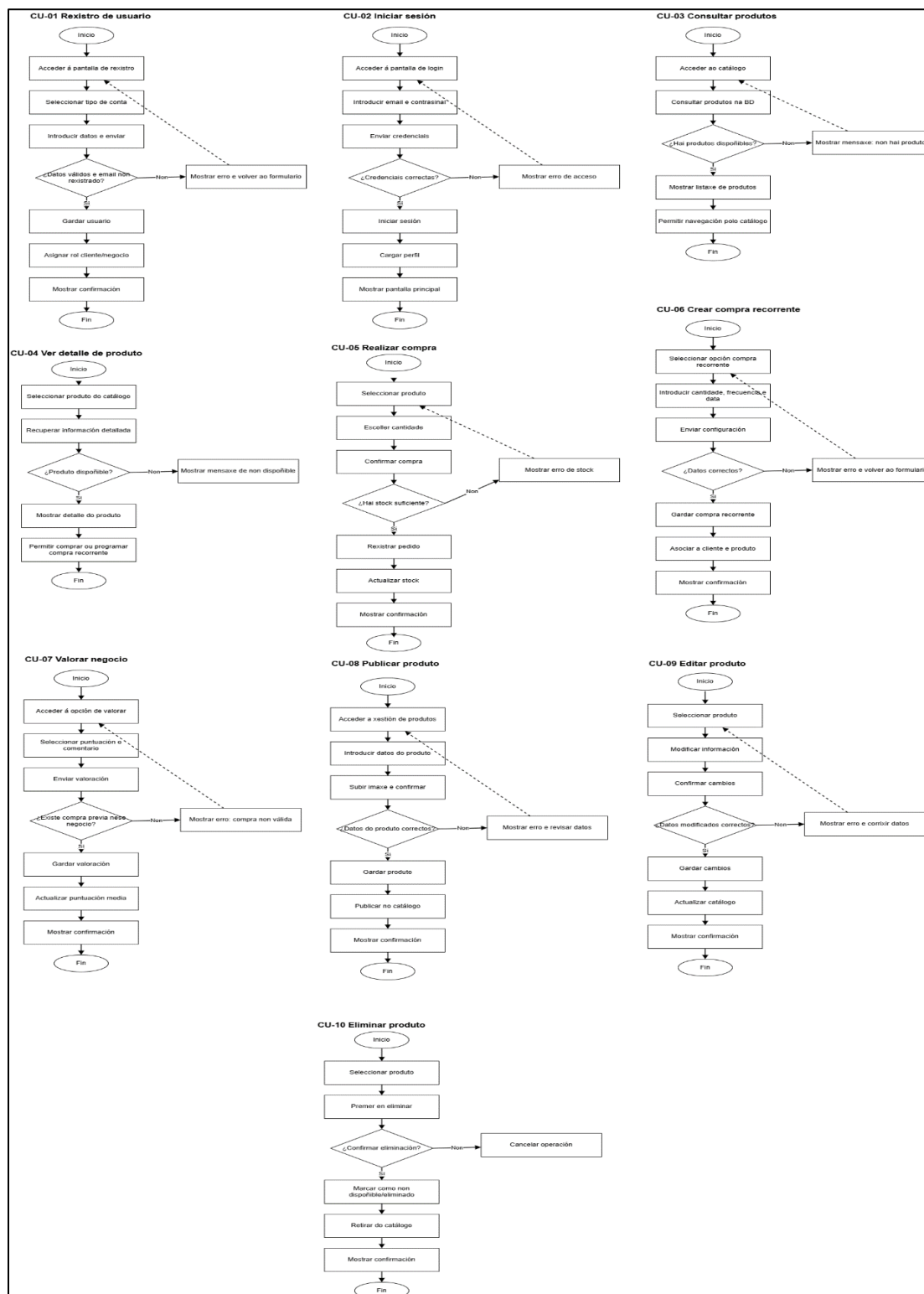


Ilustración 5 DIAGRAMA DE CASOS DE USO

Mockups da interface.

Ilustración 6 Mockups Inicio Sesión

Ilustración 7 Mockups Registro

Ilustración 8 Mockups Home Page

Ilustración 9 Mockups Detalle Producto

Ilustración 10 Mockups Compra

← Confirmar compra

Pan Artesanal
Panadería O Forno
2.80€/unidade

Cantidad
- 1 +

Resumo do pedido

Subtotal	2.80€
Gastos de envío	Gratis
Total	2.80€

Confirmar compra

Ilustración 11 Mockups Valoración

← Valorar compra

✓

Compra confirmada!
Como foi a túa experiencia?

Valoración

★★★★★

Comentario (opcional)

Compártenos a túa opinión sobre o produto...

Enviar valoración

Saltar

Ilustración 12 Mockups Página Negocio

← **Panel de Negocio**
Xestiona os teus produtos

Horta da Mariña
Pontevedra

3 Produtos 48 Vendas 4.8 Valoración

+ Engadir produto

Os meus produtos

	Tomates Gale... 3.50€/kg En stock		
	Pementos de ... 4.20€/kg En stock		
	Acelgas Frescas 1.80€/manejo Baixo stock		

Ilustración 13 Mockups Subir Producto

Subir imaxe
PNG, JPG até 5MB

Nome do produto

Ej: Tomates Galegos

Descrición

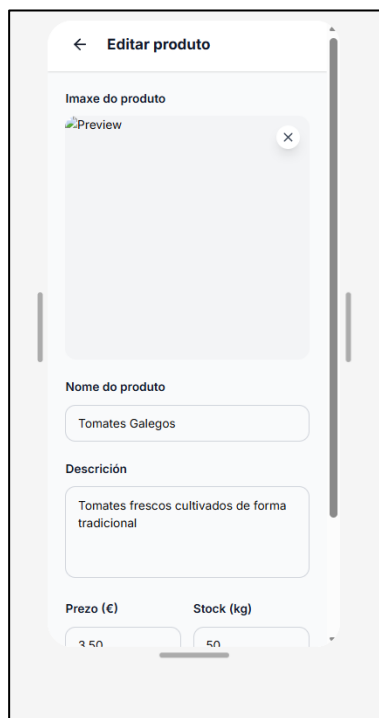
Describe o teu produto...

Prezo (€) Stock (kg)

0.00 0

Crear producto

*Ilustración 14 Mockups Editar
Producto*



← Editar produto

Imaxe do produto

Preview X

Nome do produto

Tomates Galegos

Descrición

Tomates frescos cultivados de forma tradicional

Prezo (€) Stock (kg)

2.50 50

6.6. Orzamento do proxecto

Neste apartado realízase unha estimación económica dos custos asociados ao desenvolvemento do proxecto **PrazaShop**. Para iso tivéronse en conta os recursos de hardware empregados, o software utilizado e unha estimación da man de obra necesaria para levar a cabo as distintas fases do proxecto.

Orzamento de hardware

Elemento	Cantidade	Prezo unitario (€)	Total (€)
Ordenador desenvolvemento	1	900	900
Smartphone probas	1	300	300
TOTAL HARDWARE			1200

Orzamento de software

Software	Licenza	Prezo (€)
Flutter	Gratuíto	0
Java / Spring Boot	Gratuíto	0
MySQL	Gratuíto	0
GitHub	Gratuíto	0
TOTAL SOFTWARE		0

Estimación da man de obra

Fase	Horas	Prezo/hora (€)	Total (€)
Análise	20	20	400
Deseño	20	20	400
Desenvolvemento	80	20	1600
Probas	20	20	400
TOTAL HORAS	140	TOTAL MAN DE OBRA	2800

Resumo do orzamento

Concepto	Total (€)
Hardware	1200
Software	0
Man de obra	2800
TOTAL PROXECTO	4000

O custo total estimado do proxecto **PrazaShop** é de **4000 €**. Esta cantidade inclúe os recursos materiais necesarios e a estimación económica do tempo dedicado ao desenvolvemento, mentres que o software non xera custo adicional ao empregar ferramentas gratuítas.

7. Desenvolvemento e execución

Diagrama de despregamento

O sistema PrazaShop despregase cunha arquitectura cliente-servidor. O backend está publicado en **Render** mediante un contedor Docker con **Java 17** e a aplicación **Spring Boot**. A base de datos está aloxada externamente en **Filess.io**, empregando **MySQL**.

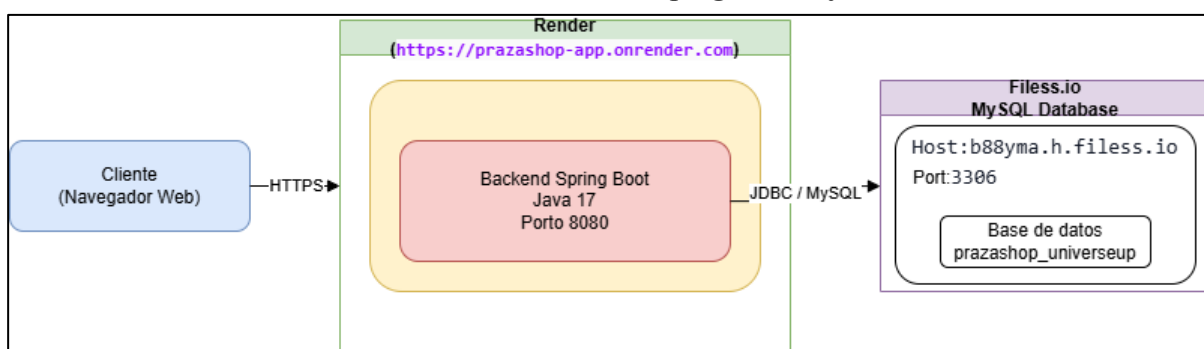


Ilustración 15 Esquema de Despegue

Definición de alcance

Funcionalidades incluídas:

- Rexistro e inicio de sesión de usuarios.
- Diferenciación entre clientes e negocios.
- Xestión de produtos por parte dos negocios.
- Consulta de produtos dispoñibles polos clientes.
- Realización de pedidos.
- Xestión do detalle dos pedidos.
- Compras recorrentes.

-
- Valoracións de negocios.

Funcionalidades fóra do alcance:

- Pasarela de pago real.
- Sistema avanzado de envíos.
- Notificacións en tempo real.
- Aplicación móbil nativa.
- Panel avanzado de administración.

O alcance do proxecto PrazaShop céntrase no desenvolvemento das funcionalidades principais dunha plataforma de comercio local: xestión de usuarios, publicación de produtos por parte dos negocios, consulta de produtos, realización de pedidos, compras recorrentes e valoracións. Quedan fóra do alcance funcionalidades avanzadas como pagos en liña reais, integración con servizos externos de transporte ou notificacións en tempo real.

8. Conclusións e reflexións

O desenvolvemento do proxecto PrazaShop permitíume aplicar de forma práctica moitos dos coñecementos adquiridos ao longo do ciclo de Desenvolvemento de Aplicacións Multiplataforma. A través deste proxecto puideron traballar no deseño e creación dunha aplicación orientada ao comercio local, combinando unha aplicación móbil desenvolvida con Flutter, unha API REST realizada con Java e Spring Boot, e unha base de datos relacional en MySQL.

Unha das aprendizaxes máis importantes foi a integración entre o backend e a base de datos, así como a definición dunha estrutura organizada para xestionar usuarios, clientes, negocios,

produtos, pedidos, compras recorrentes, valoracións, tokens e movementos de stock. Tamén me resultou útil traballar co despregamento do backend mediante Docker e Render, e coa base de datos aloxada externamente.

Entre as principais dificultades atopadas destacaríaa o deseño correcto da base de datos, xa que foi necesario pensar ben as relacións entre as entidades para evitar problemas futuros. Tamén supuxo un reto organizar todas as funcionalidades da aplicación de forma coherente, especialmente aquelas relacionadas cos pedidos, o stock e as compras recorrentes. Outro punto complexo foi diferenciar correctamente o alcance do proxecto, separando as funcionalidades principais das que quedan fóra, como a pasarela de pago real, as notificacións en tempo real ou un sistema avanzado de envíos.

9. Bibliografía e Webgrafía

Docker. (s. f.). *Docker documentation*. Recuperado o 9 de marzo de 2026, de <https://docs.docker.com/>

Filess.io. (s. f.). *The complete developer platform*. Recuperado o 10 de marzo de 2026, de <https://filess.io/>

Flutter. (s. f.). *Flutter documentation*. Recuperado o 11 de marzo de 2026, de <https://docs.flutter.dev/>

Git. (s. f.). *Git reference*. Recuperado o 12 de marzo de 2026, de <https://git-scm.com/docs>

GitHub. (s. f.). *GitHub Docs*. Recuperado o 13 de marzo de 2026, de <https://docs.github.com/>

Google. (s. f.). *Flutter: Build apps for any screen*. Recuperado o 14 de marzo de 2026, de <https://flutter.dev/>

JetBrains. (s. f.). *IntelliJ IDEA documentation*. Recuperado o 15 de marzo de 2026, de <https://www.jetbrains.com/help/idea/>

Microsoft. (s. f.). *Documentation for Visual Studio Code*. Recuperado o 16 de marzo de 2026, de <https://code.visualstudio.com/docs>

MySQL. (s. f.). *MySQL documentation*. Recuperado o 17 de marzo de 2026, de <https://dev.mysql.com/doc/>

Oracle. (s. f.). *JDK 17 documentation*. Recuperado o 18 de marzo de 2026, de <https://docs.oracle.com/en/java/javase/17/>

Postman. (s. f.). *Postman Docs*. Recuperado o 19 de marzo de 2026, de <https://learning.postman.com/>

Render. (s. f.). *Render documentation*. Recuperado o 20 de marzo de 2026, de <https://render.com/docs>

Spring. (s. f.). *Spring Boot documentation*. Recuperado o 21 de marzo de 2026, de <https://docs.spring.io/spring-boot/>

Spring. (s. f.). *Spring Data JPA reference documentation*. Recuperado o 22 de marzo de 2026, de <https://docs.spring.io/spring-data/jpa/reference/>

Fernández Bande, A. (2026). *PrazaShop-app* [Repositorio de GitHub]. GitHub. Recuperado o 30 de marzo de 2026, de <https://github.com/Abrahamfdez/PrazaShop-app>

10. Anexo I – Manual técnico de instalación ou posta en marcha

Manual técnico de instalación ou posta en marcha

1. Requisitos previos

Para instalar e executar o proxecto **PrazaShop**, é necesario dispoñer das seguintes ferramentas instaladas no equipo:

- **Git**, para descargar o repositorio do proxecto.
- **Java JDK 17**, necesario para executar o backend desenvolvido con Spring Boot.
- **Maven**, para compilar e executar o proxecto backend.
- **MySQL**, para a xestión da base de datos relacional.
- **Flutter SDK**, para executar a aplicación móbil.
- **Android Studio** ou un **emulador Android**, para probar a aplicación.
- **Visual Studio Code** ou **IntelliJ IDEA**, como contorno de desenvolvemento.
- **Postman**, para probar as chamadas á API REST.
- **Docker**, no caso de querer executar ou despregar o backend mediante contedor.

2. Descarga do proxecto

O primeiro paso consiste en descargar o código fonte desde o repositorio de GitHub:

```
git clone https://github.com/Abrahamfdez/PrazaShop-app
```

Unha vez descargado, accédese ao cartafol do proxecto:

```
cd PrazaShop-app
```

Dentro do repositorio atópanse as partes principais da aplicación:

- O **backend**, desenvolvido con Spring Boot.
- O **frontend**, desenvolvido con Flutter.

3. Configuración da base de datos

O proxecto utiliza unha base de datos relacional **MySQL**. Para a súa posta en marcha, debe crearse unha base de datos chamada, por exemplo:

```
CREATE DATABASE prazashop_universo;
```

A continuación, débense configurar os datos de conexión no ficheiro de configuración do backend, normalmente situado en:

```
src/main/resources/application.properties
```

Exemplo de configuración:

```
spring.datasource.url=jdbc:mysql://localhost:3306/prazashop_universo
spring.datasource.username=root
spring.datasource.password=contrasinal
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

No caso de empregar unha base de datos externa, como **Filess.io**, débense substituír os valores de localhost, usuario e contrasinal polos datos proporcionados pola plataforma.

4. Execución do backend

Para executar o backend, accédese ao cartafol correspondente ao proxecto Spring Boot e execútase o seguinte comando:

```
mvn spring-boot:run
```

Se a execución é correcta, o servidor iníciase no porto configurado, normalmente:

```
http://localhost:8080
```

Desde este momento, a **API REST** estará dispoñible para recibir peticións desde a aplicación móbil ou desde ferramentas como Postman.

5. Probas da API con Postman

Para comprobar o funcionamento do backend pódense realizar probas mediante **Postman**. Algúns exemplos de endpoints que se poden probar son:

```
POST /auth/register
POST /auth/login
GET /productos
POST /productos
POST /pedidos
GET /negocios
POST /valoraciones
```

Estas probas permiten verificar que o rexistro de usuarios, o inicio de sesión, a consulta de produtos, a creación de pedidos e as valoracións funcionan correctamente.

6. Execución do frontend Flutter

Para executar a aplicación móbil, accédese ao cartafol do frontend e descárganse as dependencias do proxecto:

```
flutter pub get
```

Despois, compróbase que existe un dispositivo ou emulador dispoñible:

```
flutter devices
```

Finalmente, execútase a aplicación:

```
flutter run
```

A aplicación iníciase no dispositivo ou emulador seleccionado e conéctase coa API REST configurada no backend.

7. Configuración da URL da API no frontend

Para que a aplicación Flutter poida comunicarse co backend, é necesario configurar correctamente a URL da API.

En local, a URL pode ser similar a:

```
http://localhost:8080
```

No caso de probar desde un emulador Android, pode ser necesario empregar:

```
http://10.0.2.2:8080
```

Se o backend está despregado en **Render**, débese empregar a URL pública xerada pola plataforma.

8. Execución mediante Docker

O backend tamén pode executarse mediante **Docker**. Para construír a imaxe do proxecto, utilízase o seguinte comando:

```
docker build -t prazashop-backend .
```

Unha vez creada a imaxe, execútase o contedor:

```
docker run -p 8080:8080 prazashop-backend
```

Deste xeito, o backend quedará dispoñible no porto 8080.

9. Despregamento do proxecto

O despregamento do proxecto realízase mediante unha arquitectura cliente-servidor.

O **backend** está despregado en **Render**, empregando un contedor Docker con Java 17 e a aplicación Spring Boot.

A base de datos atópase aloxada externamente en **Filess.io**, empregando MySQL.

Pasos para o despregamento en Render:

1. Subir o código fonte do backend a GitHub.
2. Crear un novo servizo en Render.
3. Conectar Render co repositorio de GitHub.
4. Configurar o despregamento mediante Docker.
5. Engadir as variables de contorno necesarias para a conexión coa base de datos.
6. Iniciar o despregamento.
7. Comprobar que a API responde correctamente desde a URL pública xerada.

10. Comprobación final

Unha vez despregado o sistema, débese comprobar o correcto funcionamento da aplicación realizando as seguintes probas:

- Rexistro dun novo usuario.
- Inicio de sesión.
- Consulta de produtos dispoñibles.
- Creación dun produto desde un usuario negocio.
- Realización dun pedido desde un usuario cliente.
- Creación dunha compra recorrente.
- Rexistro dunha valoración dun negocio.
- Comprobación dos datos almacenados na base de datos.

11. Anexo II – Documentación de uso (manuais usuario)

Manual de Usuario de PrazaShop

Versión: 0.0.1

Fecha: Mayo 2026

Tabla de Contenidos

1. Introducción
2. Primeros Pasos
 - 2.1 Instalación
 - 2.2 Registro e Inicio de Sesión
 - 2.3 Tipos de Cuentas
3. Guía para Clientes
 - 3.1 Explorar Negocios
 - 3.2 Buscar Productos
 - 3.3 Realizar una Compra
 - 3.4 Crear una Suscripción
 - 3.5 Valorar Productos y Negocios
 - 3.6 Gestionar el Historial de Pedidos
4. Guía para Vendedores
 - 4.1 Crear Perfil de Negocio
 - 4.2 Publicar Productos
 - 4.3 Monitorear Pedidos
5. Funciones Avanzadas
 - 5.1 Búsqueda y Filtros
6. Solución de Problemas
7. Preguntas Frecuentes (FAQ)
8. Contacto y Soporte
9. Índice de Figuras
10. Glosario de Términos

1. Introducción

PrazaShop es una web diseñada para conectar a clientes con negocios locales que ofrecen productos frescos, artesanales y de proximidad. Su objetivo es fortalecer el comercio local, facilitar la compra directa y promover un consumo más sostenible.

Beneficios principales:

- Acceso rápido a productos locales.
- Compras seguras y personalizadas.
- Soporte a pequeños productores y artesanos.
- Flexibilidad en compras únicas o recurrentes.
- Sistema de ofertas por tiempo limitado.

2. Primeros Pasos

2.1 Instalación

1. Abrir  **praza_shop** en un navegador.

2.2 Registro e Inicio de Sesión

1. En la pantalla principal, seleccionar **Crear cuenta**.
2. Introducir nombre, correo electrónico y contraseña.
3. Elegir tipo de cuenta: **Cliente** o **Vendedor**.
4. Pulsar **Registrarse**.

Para iniciar sesión:

1. Seleccionar **Iniciar sesión**.
2. Introducir correo y contraseña.
3. Pulsar **Entrar**.

2.3 Tipos de Cuentas

- **Cliente:** Permite explorar negocios, comprar productos y gestionar suscripciones.
- **Vendedor:** Permite administrar un negocio, publicar productos y analizar ventas.

3. Guía para Clientes

3.1 Explorar Negocios

1. Deslizar para ver negocios
2. Tocar un negocio para ver sus productos.

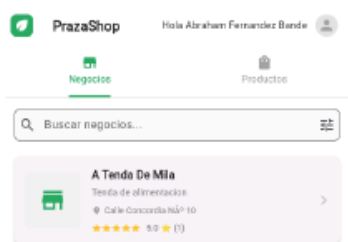


Figura 1: Pantalla de exploración de negocios (conceptual).

3.2 Buscar Productos

1. Pulsar el icono de **Búsqueda** .
2. Escribir el nombre del producto.
3. Pulsa categorías para filtrar.
4. Seleccionar un producto para ver detalles.

3.3 Realizar una Compra

1. En la ficha del producto, pulsar **comprar**
2. Confirmar con **Realizar pedido**.



Barra De Pan Campesina 1.00€

★★★★★ 5.0 (1)

in stock



A Tenda De Milla

Descripción

Barra de pan do Folto de Lito

Comprar

Compra recorrente

Confirmar compra



Barra De Pan Campesina

A Tenda De Milla

1.00€/unidade

Cantidad

-

1

+

Resumen do pedido

Subtotal 1.00€

Gastos de envío Gratis

Total 1.00€

Confirmar compra

Figura 2: Flujo de compra paso a paso (conceptual).

3.4 Crear una Suscripción

1. En el producto deseado, seleccionar **compra recorrente**.
2. Elegir frecuencia (semanal, quincenal, mensual).
3. Confirmar la suscripción.

3.5 Valorar Productos y Negocios

1. Tras la compra.
2. Seleccionar el producto o negocio.
3. Pulsar **Valorar** y asignar estrellas (1 a 5).
4. Escribir una reseña opcional.

3.6 Gestionar el Historial de Pedidos

1. Ir a **Perfil > Mis pedidos**.
2. Consultar pedidos anteriores.
3. Consulta y administra compras recurrentes
4. Repetir una compra o descargar factura.

4. Guía para Vendedores

4.1 Crear Perfil de Negocio

1. Desde la pantalla de crear conta selecciona **negocio**.
2. Completar información: nombre, dirección, descripción y Nombre del Negocio.
3. Registrarse .

4.2 Publicar Productos

1. En el panel de negocio, pulsar **Nuevo producto**.
2. Introducir nombre, descripción, precio y categoría.
3. Añadir imágenes.
4. Pulsar **Publicar**.

4.3 Monitorear Pedidos

1. Ir a **Pedidos**.
2. Ver lista de pedidos activos y completados.
3. Actualizar estado: *En preparación, Enviado, Entregado*.

5. Funciones Avanzadas

5.1 Búsqueda y Filtros

- Filtros por categoría.

6. Solución de Problemas

Problema	Posible Causa	Solución
No puedo iniciar sesión	Contraseña incorrecta	Restablecer contraseña desde la pantalla de acceso(Futura Mejora)
No se cargan productos	Conexión débil	Verificar conexión a internet

7. Preguntas Frecuentes (FAQ)

¿PrazaShop cobra comisión a los clientes?

No, el uso para clientes es gratuito.

¿Puedo vender sin tener tienda física?

Sí, siempre que se cumplan las normativas locales.

¿Cómo cancelo una suscripción?

Desde Perfil > Mis suscripciones > Cancelar.

8. Contacto y Soporte

- Correo: soporte@prazashop.com
- Teléfono: +34 900 123 456
- Horario: Lunes a viernes, 9:00–18:00
- Sitio web: www.prazashop.com

9. Índice de Figuras

Figura	Descripción
1	Pantalla de exploración de negocios
2	Flujo de compra paso a paso

10. Glosario de Términos



Término	Definición
JWT (JSON Web Token)	Sistema de autenticación segura para proteger sesiones de usuario.
Suscripción	Compra recurrente de un producto con frecuencia definida.
Dashboard	Panel de control del vendedor para gestionar su negocio.
Oferta de temporada	Promoción activa durante un periodo específico del año.
Producto artesanal	Artículo elaborado manualmente por productores locales.

PrazaShop – Conectando personas, sabores y comunidades locales.

12. Anexo III – Outra documentación

Back-end en Render:

Settings

General

Name
A unique name for your Web Service.

PrazaShop-app [Edit](#)

Region
Your services in the same [region](#) can communicate over a [private network](#).

Frankfurt (EU Central)

Instance Type

Free 0.1 CPU 512 MB [Update](#)

Please [enter your payment information](#) to select an instance type with higher limits.

[See remaining free usage](#), or learn about [free service limits](#).

Build

Source
The build source for your Web Service

<https://github.com/AbrahamIdez/PrazaShop-app> [Edit](#)

Branch
The Git branch to build and deploy.

main [Edit](#)

Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

BackEnd/Praza-Shop [Edit](#)

Registry Credential
If your service pulls private Docker images from a registry, specify a credential that can access those images. Manage your credentials in [Settings](#).

No credential [Edit](#)

Dockerfile Path
The path to your service's Dockerfile, relative to the repo root. Defaults to `./Dockerfile`.

BackEnd/Praza-Shop/ Dockerfile [Edit](#)

Docker Build Context Directory
The path to your service's [Docker build context](#), relative to the repo root. Defaults to the root.

BackEnd/Praza-Shop/ [Edit](#)

Git Credentials
User providing the credentials to pull the repository.

abrahamIdezbande@gmail.com (you) [Use My Credentials](#)

Build Filters
Include or ignore specific paths in your repo when determining whether to trigger an auto-deploy. Paths are relative to your repo's root directory. [Learn more](#).

Included Paths
Changes that match these paths will trigger a new build.

+ Add Included Path

Ignored Paths
Changes that match these paths will not trigger a new build.

+ Add Ignored Path

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

Upgrade now

May 30, 2026 at 4:00 PM

Live

766a8b7 Update ENDPOINTS_REFERENCE.md

All logs

Q Search

May 30, 3:59 PM - 4:05 PM GMT+2

⌵ ↻ ⋮

May 30

04:01:25 PM [Info] ...
04:01:25 PM [Info] (...)
04:01:25 PM [Info] (...)
04:01:25 PM [Info] (...)
04:01:25 PM [Info]
04:01:25 PM [Info] :: Spring Boot :: (v4.0.5)
04:01:26 PM [Info] 2026-05-30T14:01:26.793Z INFO 1 --- [Praza-Shop] [main] o.e.prazashop.PrazaShopApplication : Starting PrazaShopApplication v0.0.1-SNAPSHOT using Java 17.0.10 with PID 1 (/app/app.jar started by root in /app)
04:01:26 PM [Info] 2026-05-30T14:01:26.797Z INFO 1 --- [Praza-Shop] [main] o.e.prazashop.PrazaShopApplication : No active profile set, falling back to 1 default profile: "default"
04:01:47 PM [Info] 2026-05-30T14:01:47.206Z INFO 1 --- [Praza-Shop] [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
04:01:48 PM [Info] 2026-05-30T14:01:48.993Z INFO 1 --- [Praza-Shop] [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 1496 ms. Found 10 JPA repository interfaces.
04:02:05 PM [Info] 2026-05-30T14:02:05.293Z INFO 1 --- [Praza-Shop] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8000 (http)
04:02:05 PM [Info] 2026-05-30T14:02:05.593Z INFO 1 --- [Praza-Shop] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
04:02:05 PM [Info] 2026-05-30T14:02:05.593Z INFO 1 --- [Praza-Shop] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.26]
04:02:05 PM *** No open ports detected, continuing to scan...
04:02:06 PM *** Docs on specifying a port: https://xander.com/docs/web-services/port-binding
04:02:07 PM 2026-05-30T14:02:07.596Z INFO 1 --- [Praza-Shop] [main] b.w.c.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 3920ms ms
04:02:12 PM [Info] 2026-05-30T14:02:12.192Z INFO 1 --- [Praza-Shop] [main] org.hibernate.orm.jpa : HHH000540: Processing PersistenceUnitInfo [name: default]
04:02:14 PM [Info] 2026-05-30T14:02:14.192Z INFO 1 --- [Praza-Shop] [main] org.hibernate.orm.jpa : HHH000001: Hibernate ORM core version 7.2.7.Final
04:02:25 PM [Info] 2026-05-30T14:02:24.343Z INFO 1 --- [Praza-Shop] [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
04:02:25 PM [Info] 2026-05-30T14:02:24.942Z INFO 1 --- [Praza-Shop] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting..
04:02:25 PM [Info] 2026-05-30T14:02:32.893Z INFO 1 --- [Praza-Shop] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@e489b8b
04:02:32 PM [Info] 2026-05-30T14:02:32.893Z INFO 1 --- [Praza-Shop] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
04:02:34 PM [Info] 2026-05-30T14:02:34.592Z WARN 1 --- [Praza-Shop] [main] org.hibernate.orm.jpa : HHH000551: The 5.7.38 version for [org.hibernate.dialect.MySQLDialect] is no longer supported, hence certain features may not work properly.The minimum supported version is 8.0.0. Check the community dialects protect for available legacy versions.

BBDD Fless.io:

Q

Search by name, engine or region...

All

10 / page

NAME	ENGINE	REGION	STATUS	CREATED	ACTIONS
prazashop_universeup	 MySQL	Mumbai	● AVAILABLE	6/4/2026	→ Open 

Showing 1-1 of 1

< Previous

1 / 1

Next >

Databases > prazashop_universeup

 prazashop_universeup

ACTIVE

prazashop_universeup @ b88yma.h.filess.io:3306

Refresh

Web Client

OverviewResourcesBackupsConnectionsAddonsSettingsMigration

General Information

ENGINE	REGION
MySQL v5.7.38	Mumbai (ind-1)
HOST	PORT
b88yma.h.filess.io	3306

Connection Information

GeneralConnection URICLI Command

User

prazashop_universeup

Password

.....

Host

b88yma.h.filess.io

Port

3306

Database name

prazashop_universeup

Quick Actions

Kill connections

Delete database

Front-End en abrahamfernandez.me:

Actions

All workflows

Copilot coding agent

Deploy Astro to GitHub Pages

pages-build-deployment

Management

- Caches
- Deployments
- Attestations
- Runners
- Usage metrics
- Performance metrics

Deploy Astro to GitHub Pages

deploy.yml

11 workflow runs

This workflow has a workflow_dispatch event trigger.

Run workflow

Update Flutter app1	main	May 29, 11:28 AM GMT+2	33s
Update Flutter app	main	May 29, 11:16 AM GMT+2	37s
Fix Flutter app base href for GitHub Pages	main	May 28, 10:07 AM GMT+2	37s
Add initial Flutter web app files and shaders	main	May 28, 10:04 AM GMT+2	41s
style: add code-style heading for developer name	main	Mar 4, 5:39 PM GMT+1	45s
fix cv.pdf	main	Mar 4, 5:22 PM GMT+1	40s
add cv.pdf	main	Mar 4, 5:18 PM GMT+1	39s
Deploy Astro to GitHub Pages	main	Mar 4, 4:03 PM GMT+1	36s

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <https://abrahamfernandez.me/>
Last deployed by [Abrahamfdez](#) 2 days ago

Visit site

Unpublish site

Build and deployment

Source

Deploy from a branch

Branch

Your GitHub Pages site is currently being built from the main branch. [Learn more about configuring the publishing source for your site.](#)

main

/ (root)

Save

Learn how to [add a Jekyll theme](#) to your site.

Your site was last deployed to the [github-pages](#) environment by the [Deploy Astro to GitHub Pages](#) workflow. [Learn more about deploying to GitHub Pages using custom workflows](#)

Custom domain

Custom domains allow you to serve your site from a domain other than [abraham-hub-777.github.io](#). [Learn more about configuring custom domains.](#)

abrahamfernandez.me

Save

Remove

DNS Check in Progress

☒ Enforce HTTPS

HTTPS provides a layer of encryption that prevents others from snooping on or tampering with traffic to your site.

When HTTPS is enforced, your site will only be served over HTTPS. [Learn more about securing your GitHub Pages site with HTTPS.](#)